Close to optimal FIR decimator design

July 3, 2020

1 Background

Finite Impulse Reponse (FIR) filters is one of two types of digital filters, the other being Infite Impulse Response IIR filters. FIR filters can be computationally expensive but carries several advantages over IIR filters. In particular, they're always numerically stable, can have linear phase response and are simpler to design [**?**].

FIR filters are used universally in all digital medical meassuring devices such as ECG, EEG, preassured transducers etc. Because the cost of fast analog to digital converts (ADCs) and DSPs today is low it often makes sense to move as much processing as possible to the digital domain, to lower overall cost. This also gives much greater flexibilty.

Because they are computationally expensive it is of great interest to study their optimal implementation. There are several well known "tricks" for opitimization:

1.1 Decimation

A signal is frequently (*haha*) sampled at a higher sampling rate than required as this relaxes the requirements on the analog antialias filter and significally lowers cost. A band limited signal can be decimated by a factor D by simply keeping every D sample and discarding the rest. Care must be taken that the signal is sufficiently band limited or aliasing will occur. Aliasing is often unwanted but can be used for frequency translation of a bandpass signal allowing the final frequntcy correction to performed at a lower rate. In general, you always want to work at the lowest possible sample rate as this is much more efficient [**?**].

Since we only keep every D sample and discard the rest we can perform the convolution for only those samples and reduce the number of neccessary computations by D.

1.2 Half-band filters

A half-band filter is a filter whose transition region is centered as $f_s/4$ (normalized frequency $\pi/2$). Such as filter has every other coeffcient zero and the none zero coefficents are symetrical about the center of the impulse response. Both these properties can be used to improve efficiency of the implemention by transposing the computation. This is very attractive in theory but the improvents can not always be realized in practice becuause of how modern CPUs work.

A half band filter can be designed either by the window method or by using Parks–McClellan's filter design algorithm [**?**] (sometimes known as *remez*) and centering the transition band at $f_s/2$.

1.3 Fast convolution

Convolution can be performed in the frequency domain (sometimes referred to as *fast convolution*) by transforming the impulse response and input signal to the frequncy domain, multiplying them and then performing an inverse Fourier transform of the result. Since this result is circular convolution care must be taken to use sufficent length and overlap of the transform. Two techniques exist: "overlap and add" and "overlap and save". This is generally faster because the algorithmic complexity of the FFT is $O(n \log n)$ compared to convolution's $O(n^2)$. One must remember that the $O()$ complexity is the *limit* as n increases. Fast convolution is not always faster for short inputs. How short depends on the platform.

A signal can be decimated in the frequncy domain by "folding" the band limited output. Not all factors are possible, only even diviors of the length of the transform. Frequency translation is possible by shifting the transform, but again not all shifts are possible.

A disadvantage of FFT convolution is the input must be processed in blocks which can be a problem in real-time applications.

For more information see [**?**].

1.4 Multistage decimators

When decimating by an integer (none-prime) factor larger than 3, it's almost always better split the process in to multiple stages [**?**]. This might not be immeditalty obvious but considder that the first stage that has to work at the highest sampling rate can have a wider transistion band than if we used a single stage, this results in fewer taps. Actually the total number of taps for all the stages will also be less than a one stage design. In addition, following stages work at a lower rate. Because of the finite precision of floating point operations, (assuming IEEE 754) fewer operations also translates into greater accuracy. Since the previous stage will always have a transition band that's wider than follwing stage, it follows that $M_1 < M_2 < M_3 ...$ where M_n is the number of taps in stage n.

One might then ask oneself, what is the best partition (when not taking halfband optimization into consdieration)? That is the topic of the rest this paper.

1.5 Real world performance

Modern CPUs are complex machines that make heavy use of instruction pipelining and caching. A miss in branch prediction can stall the CPU for considerable time, as can access to memory not in the CPU cache. This makes it difficult to predict the real world performance, and in practice, one often has to resort to benchmarking. This is also the reason that half-band filters might not always be the best choice.

2 Designing a multistage FIR decimator

2.1 An example

There is no closed form formula for finding the optimal number of stages and the solution is to evaluate each possible combination, fortunely this is cheap to do, even for large D. There have also been attempts at finding an analytical solution [**?**].

As a demonstation we will design a multistage decimator for a signal sampled at $768ksps$ where $D = 32$, signal cut of frequncy of $5000Hz$, a transition bandwidth of $500Hz$, and a stop band attenuation of 60 dB.

This document is written as a Jupyter Notebook and hereafter code will be interwoven with the text.

[1]:
```
# First import neccessay modules
from functools import reduce
from matplotlib import pyplot as plt
from math import log10
import numpy as np

# Parks-McClellan filter design algorithm
from scipy.signal import remez

# Frequency response calculation
from scipy.signal import freqz
```

[2]:
```
# Helper function to plot frequency response of a filter.
def plot_response(fs, w, h, title, xlim = 0.5):
    "Utility function to plot response functions"
    fig = plt.figure()
    ax = fig.add_subplot(111)
    ax.plot(0.5*fs*w/np.pi, 20*np.log10(np.abs(h)))
    ax.set_ylim(-70, 5)
    ax.set_xlim(0, xlim*fs)
    ax.grid(True)
    ax.set_xlabel('Frequency (Hz)')
    ax.set_ylabel('Gain (dB)')
    ax.set_title(title)
```

First we define a function to find all the integer factors of n. Note that we only have to search to $\sqrt{n} + 1$ since if an integer i in this range is a factor of n then so is n/i.

[3]:
```
# Function to find all factors of n.
def factors(n):
    return set(reduce(list.__add__,
        ([i, n//i] for i in range(1, int(n**0.5 + 1)) if n % i == 0)))
```

```
# Example
factors(32)
```

[3]:
```
{1, 2, 4, 8, 16, 32}
```

We then define a function that returns a list of all possible combinations of stages that will decimate by n. Note that this function is recursive.

[4]:
```
# Returns list of possible integer factorizations
def factorizations(n):
    fs = []
    def fac2(n, fac=[]):
        nonlocal fs
        if n == 1:
            # we are done
            fs += [fac]
            return
        for f in factors(n)-{1}:
            fac2(n//f, fac+[f])
    fac2(n)
    return fs

# Example
factorizations(32)
```

[4]:
```
[[32],
 [2, 8, 2],
 [2, 16],
 [2, 2, 8],
 [2, 2, 2, 2, 2],
 [2, 2, 2, 4],
 [2, 2, 4, 2],
 [2, 4, 2, 2],
 [2, 4, 4],
 [4, 8],
 [4, 2, 2, 2],
 [4, 2, 4],
 [4, 4, 2],
 [8, 2, 2],
 [8, 4],
 [16, 2]]
```

We now need to design filters for each stage. To do this we must first determine the specifications for each stage and then determine the length of a filter that meets those requirements.

Let's use $[4, 4, 2]$ as an example (this might not be the optimal choice):

In the first stage the input rate of $768ksps$ has to be decimated by 4. The output rate will then be $768ksps/4 = 192ksps$. To prevent aliasing we must supress frequencies above $f_s/2 = 192kHz/2 =$

$96kHz$. We previously specified $f_c = 5kHz$, that means we can have a transition bandwidth of $96kHz - 5kHz = 91kHz$ at the first stage.

We now need to know the number of filter coefficients ("taps") required. Again there is no exact solution but there are two commonly used formulas for estimation; *Fred Harris's formula* and *Kaisers formula*. To find the actual fewest number of coeffecients that meets specification a binary search could be employed.

Note that the number of taps is inversly proportional to the width of the transition band but not the actualy cutoff frequency.

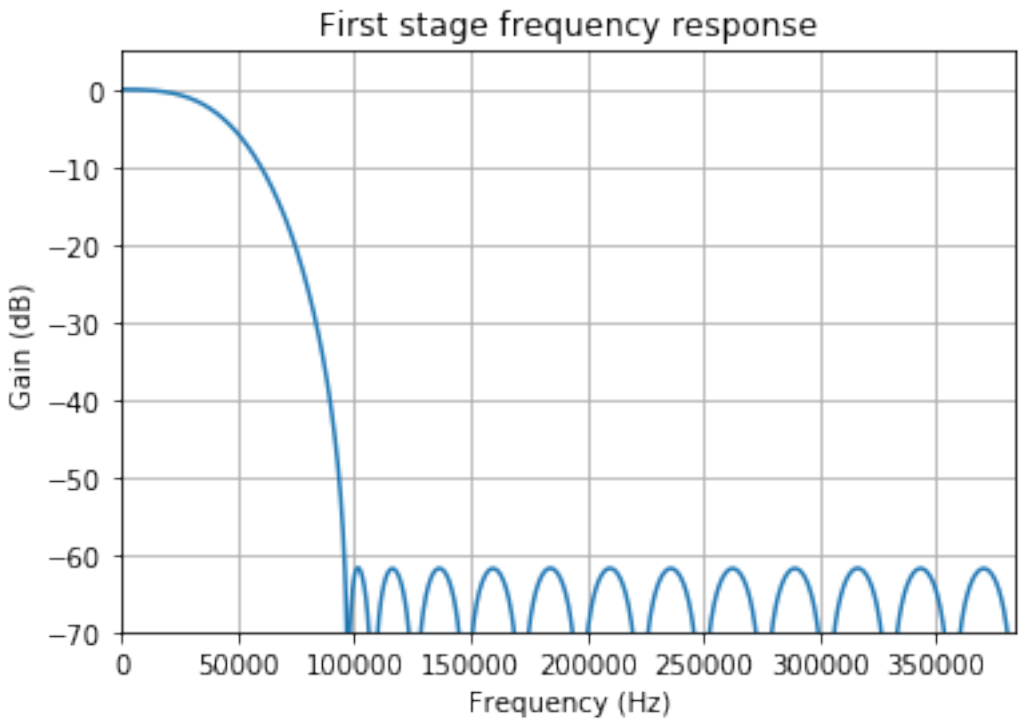
[5]:
```
# Fred Harris filter length estimation
def fred_harris(Att_dB, Trans, Fs):
    return round(Att_dB/22*Fs/Trans)

# Kaisers filter length estimation
def kaisers(Ripple_dB, Att_dB, Trans, Fs):
    ripple = 1 - 10**(-Ripple_dB/20)
    att = 10**(-Att_dB/20)
    length = (-10*log10(ripple*att)-13)/(14.6*(Trans/Fs))
    return round(length)

# Example:
# Estimate how many taps are required for a filter with a transistion bandwidth␣
 ↪of 91000Hz
# at a sample rate of 768000Hz. (0.5dB ripple is a common choice).

fred_harris(60, 91000, 768000) # stage 1
```

[5]:
```
23
```

As we see from above calculation such *Fred Harris's* estimates such a filter to need 23 taps and *Kaiser's* 17 taps. Let's try to design such a filter using *Parks–McClellan's* (remez) algorithm (The *Parks–McClellan* algorithm, published 1972, is an iterative algorithm for finding the optimal Chebyshev FIR filter).

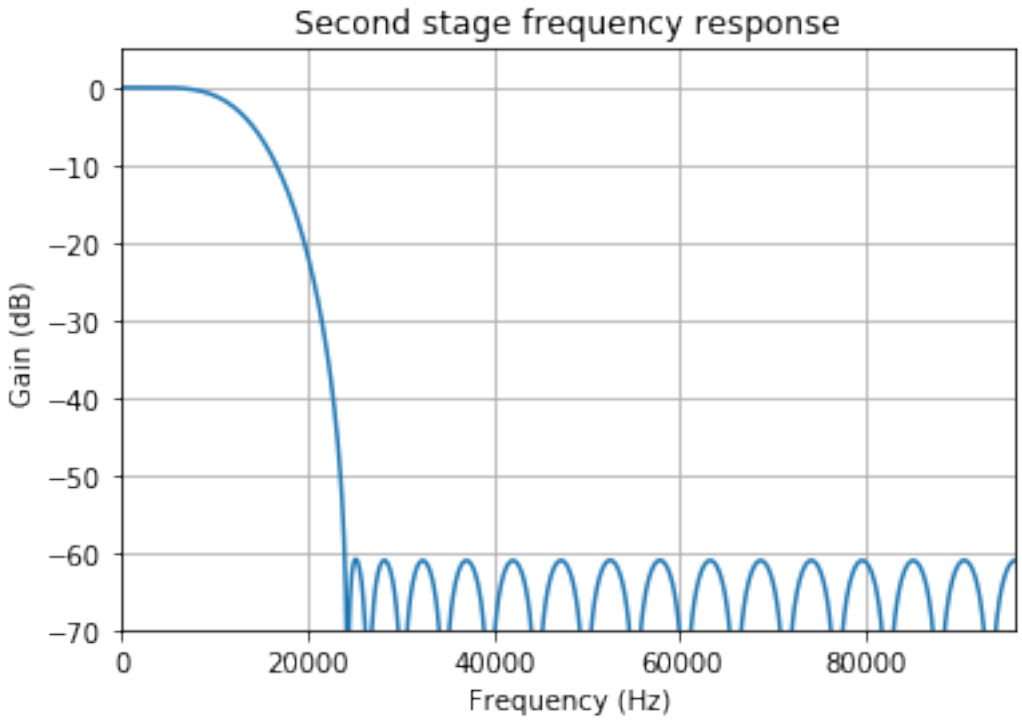
Since these are estimation we have to actually design a filter and look at the result. Using 23 gives us a stopband supression of $-50dB$, by experimentation we find that the number of taps actually needed is 28.

[6]:
```
# First stage
fs = 768000
s1_taps = remez(28, [0, 5000, 96000, fs/2], [1, 0], Hz=fs) # 23
w, h = freqz(s1_taps, [1], worN=1024)
plot_response(fs, w, h, "First stage frequency response")
```



Let's continue the second stage. Our input signal is now decimated to $192,000ksps$. We again decimate by 4, output rate will be $192,000/4 = 48,000ksps$, transition bandwidth can be $24000 - 5000 = 19000Hz$ and estimated taps calculated 28. Again we experiment and find that 35 taps are neccessary.

[7]:
```
# Second stage
fs = 192000
s2_taps = remez(35, [0, 5000, 24000, fs/2], [1, 0], Hz=fs) #28
w, h = freqz(s2_taps, [1], worN=1024)
plot_response(fs, w, h, "Second stage frequency response")
```



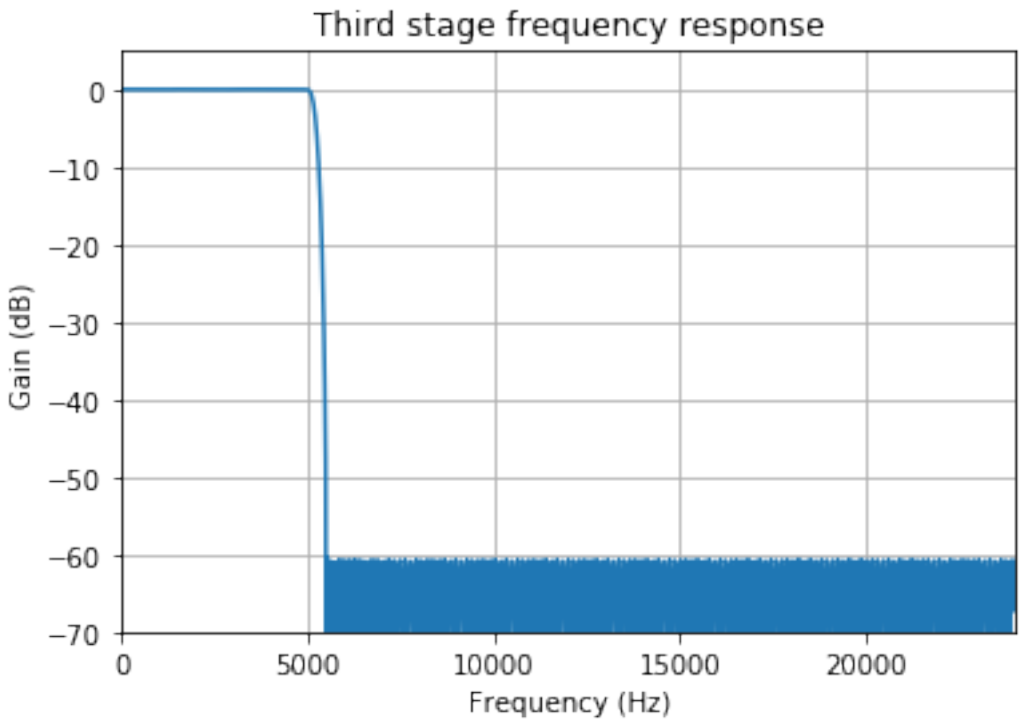
At our third stage we an input rate of $48ksps$ that will decimate by a factor of 2. Since this is our final stage our transition band will have to be $500Hz$ as we specified in the requirements. Again we estimate the nubmer of taps needed and then expriment, for $-60dB$ we find the number of taps needed to about 320.

Note: Modern CPUs use SIMD instructions. It can often be an advantage to use a filter length thats a multiple of the CPU vector length.

[8]:
```
fred_harris(60, 500, 48000) # stage 1
```

[8]:
```
262
```

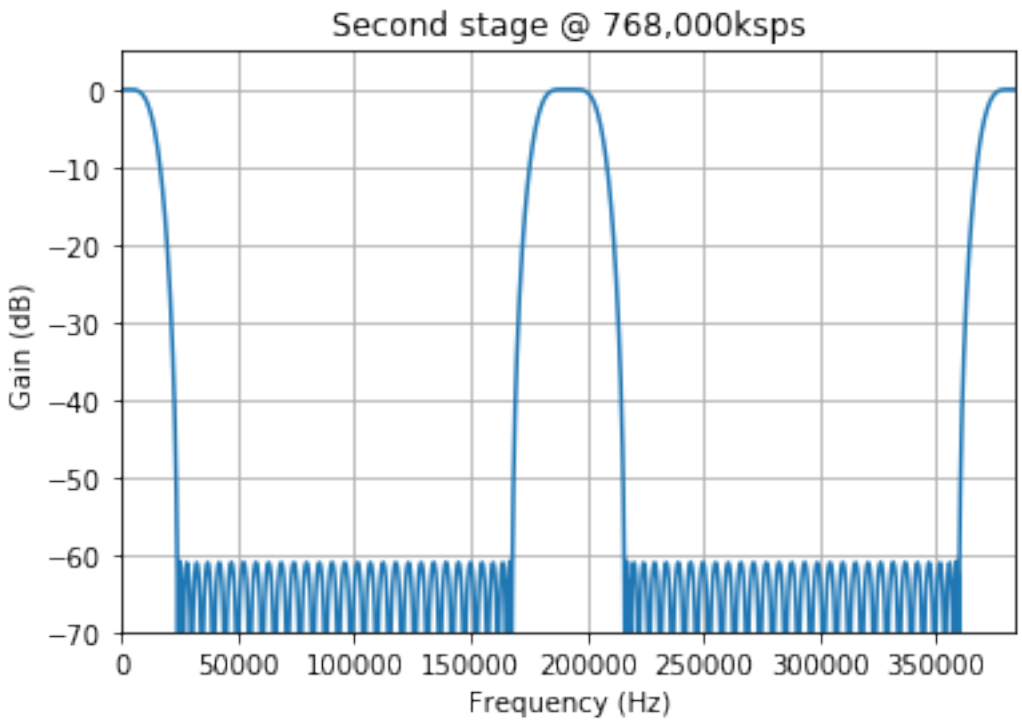
[9]:
```
# Third stage
fs = 48000
s3_taps = remez(320, [0, 5000, 5500, fs/2], [1, 0], Hz=fs)
w, h = freqz(s3_taps, [1], worN=1024)
plot_response(fs, w, h, "Third stage frequency response")
```



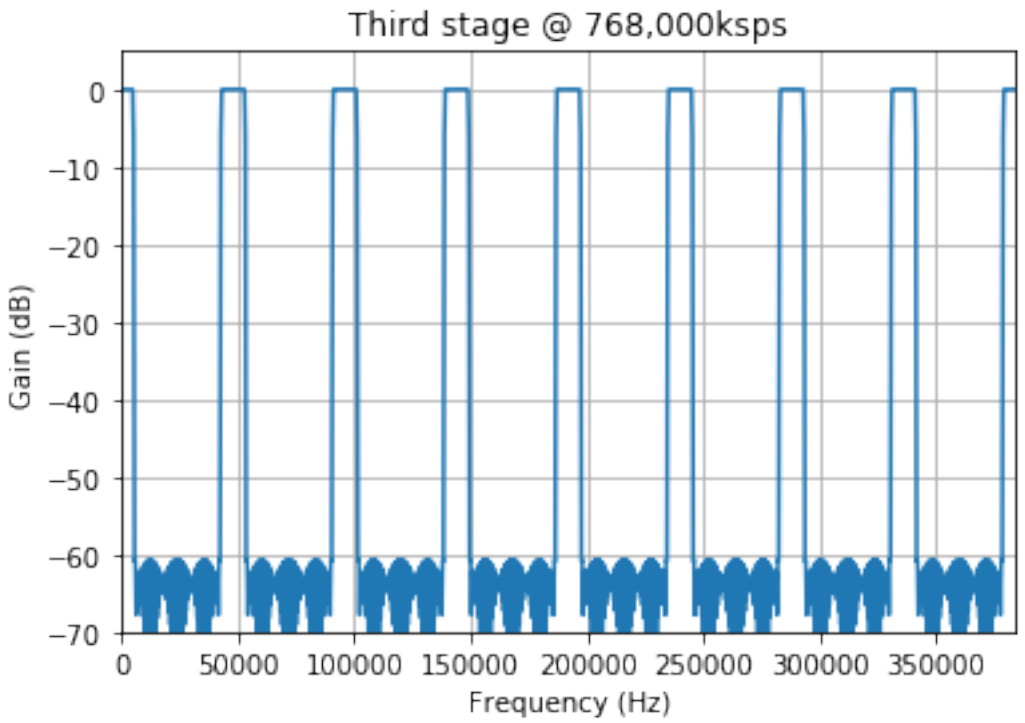
We have now defined and verified taps for our three stages. What is the combined frequeny response for all three stages?

To determine the combined response we must upsample the last two stages so that all stages are sampled at the same rate; 768*ksps*. Looking at the result we see the expected peeks at multiples of the original sample rate.

[10]:
```
# Helper function to insert n-1 zeros between samples
def upsample(array, n):
    new_array = [0]*(n*len(array)-(n-1))
    new_array[::n] = array
    return np.array(new_array)

# Upsample s2 taps by 4 from 192,000 to 768,000ksps
s2_upsampled = upsample(s2_taps, 4)

fs = 768000
w, h = freqz(s2_upsampled, [1], worN=1024)
plot_response(fs, w, h, "Second stage @ 768,000ksps")
```

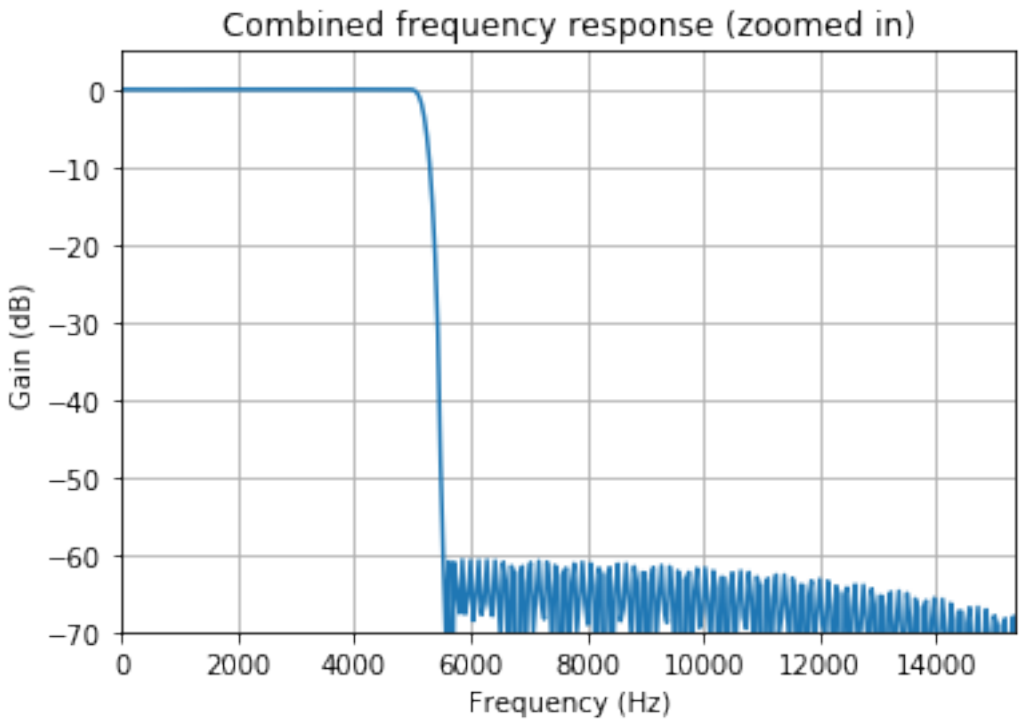


[11]:
```
s3_upsampled = upsample(s3_taps, 16)

fs = 768000
w, h = freqz(s3_upsampled, [1], worN=1024)
plot_response(fs, w, h, "Third stage @ 768,000ksps")
```



The combined frequency response can then be determined by convolving the taps $S_c = S_1 \circledast S_2 \circledast S_3$.

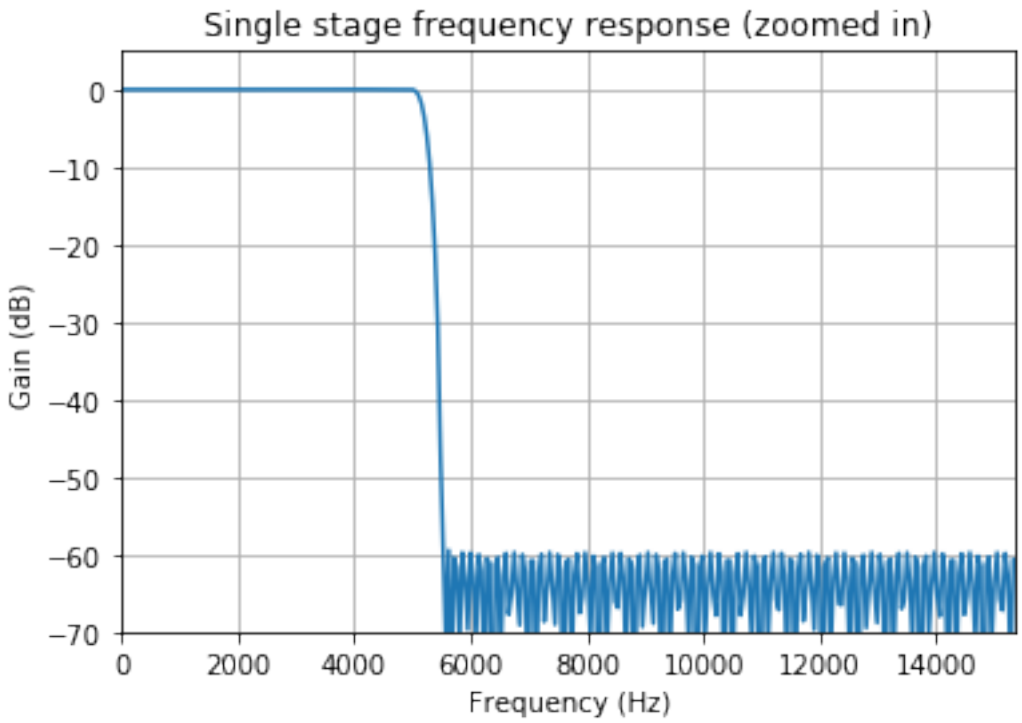
[12]:
```
s123_combined = np.convolve(s1_taps, np.convolve(s2_upsampled, s3_upsampled))

fs = 768000
w, h = freqz(s123_combined, [1], worN=8192)
plot_response(fs, w, h, "Combined frequency response (zoomed in)", xlim=0.02)
```



As can be seen in above figure the combined frequency response of our multistage decimator is almost what we expect but we also have to closely examine the ripple in the passband to confirm that it's also within specification.

Below you see a single stage filter of similar performance. Notice how we need to use 5000 taps. In our multistage design we only used $28 + 35 + 320 = 383$.

[13]:
```
# Single stage
fs = 768000
single_taps = remez(5000, [0, 5000, 5500, fs/2], [1, 0], Hz=fs)
w, h = freqz(single_taps, [1], worN=8192)
plot_response(fs, w, h, "Single stage frequency response (zoomed in)", xlim=0.
↪02)
```



2.2 Computational savings

We have shown how to design a multistage decimator, but what computational savings can we expect?

Since each step of the convolution involves calculating the dot product, a convenient meassure is the *multiplications and adds per second* (MADS). For example, a filter with M taps requires M multplications and $M-1$ additions per output sample or approximately $f_s * M/D$ MADS (we only need to calculate the output for every D sample). In our example we downsampled by 32 in three steps $[4, 4, 2]$ using filters of lengths $[28, 35, 320]$.

Multi stage, MADS: $(768000 * 28)/4 + (192000 * 35)/4 + (48000 * 320)/2 = 14736000$ MADS.

Single stage, MADS: $768000 * 5000/32 = 120000000$ MADS.

Relative savings: $14736000/120000000 = 0.1228$

To conclude our example: Our three stage design achives better result than a one stage design and only uses 12.28% of the computations.

2.3 Searching for the best factorization

As we have demonstrated, a multistage FIR decimator can be considerably more efficient than a single stage decimator. But how do we chose the best number of stages? A function `cost` calculates

the cost in MADS for all possible factorizations and print the relative cost defined as MADS for a multistage decimator divided by MADS for single stage decimator.

By inspecting the result we find that the best option in terms of MADS is $[16, 2]$.

```
# Best choice
D:16 fs:768000 est_taps:110
D: 2 fs: 48000.0 est_taps: 262
cost: 11568000.0 MADS
relative cost: 0.11506326092146096

# Our example
D:4 fs:768000 est_taps:23
D:4 fs:192000.0 est_taps:28
D: 2 fs: 48000.0 est_taps: 262
cost: 12048000.0 MADS
relative cost: 0.11983767008832658
```

[14]:
```python
def cost(fc, final_trans_bw, fs, decs, log=True):
    total_cost = 0
    last_dec = decs.pop()
    for d in decs:
        stage_out_bw = fs/2/d
        transistion_bw = stage_out_bw-fc
        est_taps = fred_harris(60, transistion_bw, fs)
        cost = est_taps*fs/d
        total_cost = total_cost + cost
        if log: print("D:{} fs:{} est_taps:{}".format(d, fs, est_taps))
        fs = fs/d
    est_taps = fred_harris(60, final_trans_bw, fs)
    cost = est_taps*fs/last_dec
    total_cost = total_cost + cost
    if log: print("D: {} fs: {} est_taps: {}".format(last_dec, fs, est_taps))
    if log: print("cost: {} MADS".format(total_cost))
    return total_cost

# Our previous experiment cost: 14736000
cost(5000, 500, 768000, [32])
cost(5000, 500, 768000, [4,4,2])
```

```
D: 32 fs: 768000 est_taps: 4189
cost: 100536000.0 MADS
D:4 fs:768000 est_taps:23
D:4 fs:192000.0 est_taps:28
D: 2 fs: 48000.0 est_taps: 262
cost: 12048000.0 MADS
```

[14]:
```
12048000.0
```

[17]:
```
def cost_all(fc, final_trans_bw, fs, D):
    rel = cost(fc, final_trans_bw, fs, [D], log=False)
    for d in factorizations(D):
        c = cost(fc, final_trans_bw, fs, d)
        print("relative cost: {}\n".format(c/rel))

# Print total and relative cost for all possible factorization
cost_all(5000, 500, 768000, 32)
```

```
D: 32 fs: 768000 est_taps: 4189
cost: 100536000.0 MADS
relative cost: 1.0

D:2 fs:768000 est_taps:11
D:8 fs:384000.0 est_taps:55
D: 2 fs: 48000.0 est_taps: 262
cost: 13152000.0 MADS
relative cost: 0.13081881117211744

D:2 fs:768000 est_taps:11
D: 16 fs: 384000.0 est_taps: 2095
cost: 54504000.0 MADS
relative cost: 0.5421341608975889

D:2 fs:768000 est_taps:11
D:2 fs:384000.0 est_taps:12
D: 8 fs: 192000.0 est_taps: 1047
cost: 31656000.0 MADS
relative cost: 0.31487228455478633

D:2 fs:768000 est_taps:11
D:2 fs:384000.0 est_taps:12
D:2 fs:192000.0 est_taps:12
D:2 fs:96000.0 est_taps:14
D: 2 fs: 48000.0 est_taps: 262
cost: 14640000.0 MADS
relative cost: 0.1456194795894008

D:2 fs:768000 est_taps:11
D:2 fs:384000.0 est_taps:12
D:2 fs:192000.0 est_taps:12
D: 4 fs: 96000.0 est_taps: 524
cost: 20256000.0 MADS
relative cost: 0.20148006684172834

D:2 fs:768000 est_taps:11
D:2 fs:384000.0 est_taps:12
```

```
D:4 fs:192000.0 est_taps:28
D: 2 fs: 48000.0 est_taps: 262
cost: 14160000.0 MADS
relative cost: 0.14084507042253522

D:2 fs:768000 est_taps:11
D:4 fs:384000.0 est_taps:24
D:2 fs:96000.0 est_taps:14
D: 2 fs: 48000.0 est_taps: 262
cost: 13488000.0 MADS
relative cost: 0.13416089758892338

D:2 fs:768000 est_taps:11
D:4 fs:384000.0 est_taps:24
D: 4 fs: 96000.0 est_taps: 524
cost: 19104000.0 MADS
relative cost: 0.1900214848412509

D:4 fs:768000 est_taps:23
D: 8 fs: 192000.0 est_taps: 1047
cost: 29544000.0 MADS
relative cost: 0.2938648842205777

D:4 fs:768000 est_taps:23
D:2 fs:192000.0 est_taps:12
D:2 fs:96000.0 est_taps:14
D: 2 fs: 48000.0 est_taps: 262
cost: 12528000.0 MADS
relative cost: 0.12461207925519217

D:4 fs:768000 est_taps:23
D:2 fs:192000.0 est_taps:12
D: 4 fs: 96000.0 est_taps: 524
cost: 18144000.0 MADS
relative cost: 0.1804726665075197

D:4 fs:768000 est_taps:23
D:4 fs:192000.0 est_taps:28
D: 2 fs: 48000.0 est_taps: 262
cost: 12048000.0 MADS
relative cost: 0.11983767008832658

D:8 fs:768000 est_taps:49
D:2 fs:96000.0 est_taps:14
D: 2 fs: 48000.0 est_taps: 262
cost: 11664000.0 MADS
relative cost: 0.11601814275483409
```

```
D:8 fs:768000 est_taps:49
D: 4 fs: 96000.0 est_taps: 524
cost: 17280000.0 MADS
relative cost: 0.1718787300071616

D:16 fs:768000 est_taps:110
D: 2 fs: 48000.0 est_taps: 262
cost: 11568000.0 MADS
relative cost: 0.11506326092146096
```